

SISTEMAS PARALELOS Y DISTRIBUIDOS. 3º GIC. PRÁCTICA 9.

OPENMP Y MODELO DEL TEJADO.

OBJETIVOS Y PREPARACIÓN

En la actualidad los multiprocesadores en un solo chip (denominados habitualmente procesadores de múltiple núcleo) son los predominantes en el mercado. Si queremos aprovechar las posibilidades de estos procesadores es necesario particionar nuestro código en diversos hilos de forma que puedan ejecutarse simultáneamente tantos hilos como procesadores tengamos. Con el fin de facilitar este proceso se han introducido herramientas que permiten realizar esta partición de manera sencilla sin tenerse que preocupar de crear y destruir los hilos o de gestionar las comunicaciones entre ellos de forma explícita. Dentro de estas herramientas una de las más populares es el entorno openMP que hoy en día está soportado en la mayoría de multiprocesadores con memoria compartida. En la familia 80x86 openMP están soportados los compiladores de C++ de Microsoft e Intel (Windows y Linux) y otros. La referencia de openMP podemos encontrarla en <http://openmp.org/wp/>.

En esta práctica se estudiarán primero diversos códigos que permitirán comprender el funcionamiento de OpenMP y de la programación de hilos en paralelo. Después se estudiará el límite de rendimiento usando tanto OpenMp como núcleos vectoriales, con objeto de acercarse al llamado Modelo del Tejado (Roofline Model).

Como primera **preparación se deben** repasar las principales directivas de OpenMP. Piensa teóricamente como sería el código SAXPY usando la directiva y las funciones siguientes y escribirlo en la tabla final (TABLA DE PREPARACIÓN). Usar la directiva **#pragma omp parallel** y dentro de ella, reescribir el bucle SAXPY, de manera que cada procesador ejecute una parte igual del bucle. Para ello, use las funciones:

- `omp_get_num_threads ()` para obtener el número de hilos creados N_t , y
- `omp_get_thread_num ()` para obtener el identificador de hilo ($tid=Thread ID$).

Pista: la idea es que cada hilo ejecute el mismo bucle pero cada uno con distintos datos, es decir, cada hilo tid ejecuta desde $i=tid*tam/N_t$ a $i=(tid+1)*tam/N_t$.

Ver las siguientes transparencias del tema 5:

- “Cálculo de PI: Omp “manualmente”,
- “Sincronización: Barreras (I)”

Después se trabajará con los siguientes pequeños bucles. Para ellos, se pide que se **rellene la segunda parte de la tabla de preparación**. Dibuje el modelo del tejado (variables tipo *float*) para los PCs del F0.33, para el ancho de banda de la RAM y para el ancho de banda de las cachés L1, L2 y L3, sabiendo que:

CPU: Intel(R) Core(TM) i7-8400 CPU @ 3.19 GHz. 6 cores (12 hilos). 3 UF vectoriales AVX2 con registros de 256 bits

AB (L1) = 1536.1 GB/s

AB (L2) = 414.38 GB/s

AB (L3) = 329.24 GB/s

AB (RAM) = Single 17.98 GB/s. Tiene DDR4 1200.3 MHz single channel.

Tam (L1) = 6x32 KB (6 caches, uno por core). Tam (L2) = 6x256 KB (6 caches). Tam (L3) = 1x12 MB (1 cache)

DA = densidad aritmética = (Núm FLOP) / (núm de bytes accedidos).

Se trabajará con estas funciones (cada una por separado):

```
float w1[MAXSIZE], w2[MAXSIZE], w3[MAXSIZE], w4[MAXSIZE], w5[MAXSIZE], w6[MAXSIZE];
float b0 = 2, b1 = 3;
void low_DA (int n) {
    for (int i = 0; i < n; i++) {
        w1[i] = w2[i] * w3[i];
        w4[i] = w5[i] + w6[i];
    }
}
void high_DA (int n) {
    for (int i = 0; i < n; i++) {
        w2[i] = b0 * w2[i] * w2[i] * w2[i] + b1;
    }
}
float summation (int n) {
    float aux, sum=0;
    for (int i = 0; i < n; i++) {
        aux = w6[i] * w5[i];
        sum += aux;
    }
    return sum;
}
```

```
int z[MAXSIZE]; // z[] contiene 0 y 1 aleatoriamente
int countzeros (int n) {
    int count1=0;
    for (int i = 0; i < n; i++) {
        if (z[i] <= 0)
            count1++;
    }
    return count1;
}
```

REALIZACIÓN DE LA PRÁCTICA.

EJERCICIO 1. Abre la solución **p6**. Dentro de ella elige el proyecto **ej1** para que sea el proyecto inicial y abre dentro de él, el fichero **principal_ej1.cpp**, que contiene este código:

```
using namespace std;

#include <iostream>
#include <omp.h>

int main()
{
    cout << "Hola, mundo!" << endl;
}
```

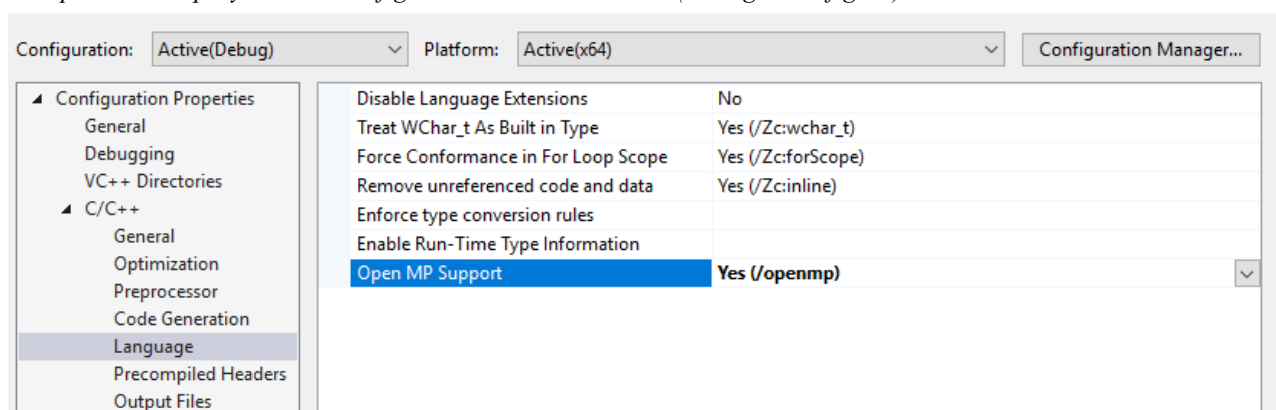
a) Lo más sencillo que se le puede pedir a OpenMP es que cree diferentes hilos y ejecute en todos ellos un mismo código. Cambia la línea `cout ...` en `main()` por esta otra:

```
#pragma omp parallel
{
    cout << "Hola, mundo!" << endl;
}
```

Si el sistema en el que se está ejecutando este programa contiene dos o más procesadores (lógicos o físicos), se habrán creado tantos hilos de ejecución paralela como procesadores existan, así que deberían aparecer tantos mensajes “Hola, mundo!” como hilos se hayan creado.

Si sólo aparece un mensaje, y nuestro sistema es multi-core, probablemente signifique que nuestro compilador no soporta directivas OpenMP, o bien que éstas no han sido habilitadas en:

Propiedades de proyecto -> Config. -> C/C++ -> Idioma (ver siguiente figura)



b) Se puede forzar un número de hilos determinado usando la siguiente función del API de OpenMP: `omp_set_num_threads (numhilos);` Cambia el código para crear 4, 8 ó 16 hilos.

```
int main()
{
    omp_set_num_threads(4); // crea 4 hilos para cada pragma omp parallel que se encuentre
    #pragma omp parallel
    {
        cout << "Hola, mundo!" << endl;
    }
}
```

Tanto en este caso como en el anterior, es más que posible que los mensajes en pantalla aparezcan “pisados” unos con otros. Esto es porque las múltiples llamadas a *cout* se ejecutan a la vez. Lo solucionaremos en un instante.

c) Habitualmente nos interesa que cada uno de los hilos que se crea sea consciente de que es un hilo dentro de un conjunto de hilos que se ejecuta en paralelo. Para ello hay dos funciones del API de OpenMP:

`int omp_get_thread_num()` : esta función devuelve el número de hilo en el que se encuentra.

`int omp_get_num_threads()` : esta función devuelve el número de hilos que se están ejecutando en paralelo en este momento.

Cambia el código para que el mensaje que se muestra por pantalla indique qué hilo se está ejecutando y cuántos hay. Es decir, que aparezcan líneas con un texto tal como: *Hola, mundo! Soy el hilo __ de __.*

A continuación vamos a resolver los mensajes en pantalla “pisados” unos con otros.

d) Busca información sobre la directiva `#pragma omp critical` y úsala para arreglar el programa obtenido en c), de manera que los mensajes salgan correctamente.

EJERCICIO 2.

Ahora vamos a usar el proyecto **ej2**, en la misma Solución. Activa **ej2** y márcalo como proyecto de inicio. Abre dentro de él, el fichero **principal_ej2.cpp**.

a) Queremos paralelizar este bucle. Es decir, que varias iteraciones del mismo se ejecuten en diferentes hilos. Para ello se ha añadido una directiva `#pragma omp parallel` de forma que el bloque que contiene el bucle se ejecute concurrentemente en varios hilos. Queremos que se ejecute el mismo número de iteraciones, pero cada una de ellas (o un grupo de ellas) en hilos diferentes. Prueba primero el programa comentando el `#pragma` y comprueba que el número de iteraciones que produce es el esperado. Descomenta el `#pragma` para que surta efecto ¿Qué resultado tenemos ahora? NOTA: evitar que los resultados en pantalla se pisen usando `#pragma omp critical`

b) Usa las funciones `omp_get_thread_num()` y `omp_get_num_threads()` para particionar el bucle de la siguiente forma: el bucle tiene n iteraciones. Sean p hilos, donde p es el valor devuelto por `omp_get_num_threads()`. Supondremos que el número de iteraciones n es divisible entre el número de hilos p , o sea, $n \% p = 0$.

Entonces, para cada hilo t ($0 \leq t < p$), el valor inicial del bucle *for* será $t*(n/p)$ y el valor final (al cual no debe llegar el bucle), $(t+1)*(n/p)$. Comprueba que el número de iteraciones es el correcto tanto si el *pragma* existe como si no.

c) Usando `#pragma omp parallel for` escribe de otra forma el programa del apartado b), para que se obtenga la misma salida por pantalla

EJERCICIO 3.

Activa el proyecto **ej3**, márcalo como proyecto de inicio y dentro de él, abre el fichero **principal_ej3.cpp**

Este programa cronometra el tiempo de ejecución de la función que se ponga en el sitio indicado. Se trata de comparar el rendimiento en ciclos por iteración del código C en GFLOPS, usando los `#pragma omp` correctos.

Antes de ejecutar:

- 1) Escriba en el código el valor de la variable `flop_iter_C`; con el número de FLOP por iteración del bucle en C (ver preparación) tras cada llamada (en el código que se da, todas tienen valor 0)
- 2) Ponga los `omp pragma` adecuados. Se recomienda usar `default (none)` y declarar el ámbito de toda variable (*private, shared, reduction, etc*)

Al ejecutar compruebe que el resultado de cada función es el correcto. Atención a la variable `aux` de `summation()`.

Con los datos obtenidos, rellene la tabla de este boletín. Para no tener que rellenar todos los datos que salen por consola, elija para cada bucle solo el mejor de entre:

- Aquellos tamaños que caben en L1 multicore
- Aquellos tamaños que caben en L2 multicore pero no en L1
- Aquellos tamaños que caben en L3 pero no en L2
- Aquellos tamaños que no caben en L3, y acceden a memoria

Justifique valores de rendimiento para cada tamaño de vector.

Finalmente ponga un punto de ruptura en las funciones `high_DA_omp_asm_only_float (n); high_DA()` , `countzeros_condit_processing(n);` comparado con `countzeros()`, con objeto de ver como es el ensamblador del bucle.

Justifique por qué las dos funciones alternativas dan mejores resultados para ciertos tamaños de los vectores y no para otros. Es decir:

`high_DA_omp_asm_only_float (n);` comparado con `high_DA()`
`countzeros_condit_processing(n);` comparado con `countzeros()`

Página dejada intencionadamente en blanco

SISTEMAS PARALELOS Y DISTRIBUIDOS. PRÁCTICA 9

ALUMNO:

TABLA DE PREPARACIÓN

Código SAXPY reescrito con `omp_get_num_threads()` y `omp_get_thread_num()`

BUCLE	Num FLOP por iter C	D _A	Núm de ciclos por iteración si el único cuello de botella fuera el grado superescalaridad m=4. Cada bucle tiene tres instrucciones de overhead
low_da			
high_da			
summation			
countzeros			Para este suponer que BTB siempre acierta

Dibujar modelo del tejado (variables tipo float), y ubicar los bucles anteriores

TABLA DE RESULTADOS

EJERCICIO 1.

Código apartado c)

Código apartado d)

EJERCICIO 2.

Código apartado b)

Código apartado c)

EJERCICIO 3.

Número de ciclos aprox. de overhead de creación y destrucción de hilos en OpenMP (justificar para “size” pequeños):

(usar solo dos decimales para cada valor).

	Low DA		High DA		Summation		Count zeros	
Size	cycles per iter C	GFLOPs	cycles per iter C	GFLOPs	cycles per iter C	GFLOPs	cycles per iter C	GFLOPs
cabe en L1								
cabe en L2								
cabe en L3								
No cabe en L3								

JUSTIFICACIÓN DE LOS VALORES DE LA TABLA

Low_DA (indicar también en qué porcentaje se acerca al ideal del modelo del tejado)

High_DA (indicar también en qué porcentaje se acerca al ideal del modelo del tejado)

Summation (indicar también en qué porcentaje se acerca al ideal del modelo del tejado)

Count_zeros

JUSTIFICACIÓN DE LOS VALORES DE LOS BUCLES COMPARANDO CON LAS VERSIONES

high_DA_omp_asm_only_float (n); comparado con high_DA()
(indicar también en qué porcentaje se acerca al ideal del modelo del tejado)

countzeros_condit_processing(n); comparado con countzeros()